CPSC 335 - RescueRoute

RescueRoute is a Python class project that matches surplus food donations with community groups and volunteer drivers. It compares FIFO and greedy scheduling. Each donation gets an assignment or a reason why it could not be assigned.

All sample data is made up for this project. This is a class model, not a food-safety system or a tool approved for real pickup work.

Setup and usage

Use Python 3.12 to match the version checked for this project (Python 3.12.3). The program and tests use Python's built-in tools. No extra packages are needed. Git is needed only if you want to clone the repository.

The project repository currently provides sample data that can be accessed in the 'data' folder, adhering to the provided format as well as any sample locations/areas that are described in the current lists.

This program runs as a command-line Python application where it displays all relevant output in the command-line-interface when you run the project by the directions that are listed below:

Download the repository, or clone it:

```
git clone https://github.com/dyaseer/CPSC335-Rescue-Route
cd CPSC335-Rescue-Route
```

From the project folder, run:

```
python main.py
```

On Windows, use `py` instead of `python` if that is how Python is installed. On macOS or Linux, you may need `python3`.

The program reads the three JSON files in `data/`. It prints FIFO results, greedy results, and a comparison of their totals. Each donation has a result and a reason. Assigned donations also show the recipient, driver, and pickup time. Greedy results explain the priority order.

For example, the current FIFO output includes:

```
[ASSIGNED] Donation D-001
Donation D-001 was assigned to Community Shelter with vol
Scheduled pickup: minute 50 of the simulation.
```

To try different inputs, edit the JSON files in `data/` and run again. Each file must contain a list of records. Use `[]` for an empty list. A blank file is not valid JSON. Invalid data stops the program with an error. There are no command-line options for choosing other input files.

Tests

If you would like to run the dedicated unit tests for this project, you may run the following command:

```
python -m unittest discover -s tests -v
```

All 12 tests passed with no skipped tests on October 14, 2026. The tests check:

1. Volunteer carrying-space use, including multiple pickups and zero capacity.
2. A normal match with the right recipient, driver, and pickup time.
3. Duplicate donation IDs.
4. Donations expiring before, at, and just after pickup time.
5. A recipient that does not accept the food type.
6. Greedy order and its explanation.
7. Too little recipient space and an exact-capacity match.

8. Small vehicles, unavailable drivers, and drivers with no pickups left.
9. Donations competing for space or pickups, without changing original inputs.
10. Empty lists, a JSON file containing `[]`, an invalid blank file, and zero rates.
11. Different FIFO and greedy choices on the same original data.
12. A later, larger donation getting priority over an earlier, smaller one.

Tests check expected results and reasons, rather than only whether the program runs.

Test case with different results

`test_fifo_and_greedy_differ` uses its own data, separate from the sample files. D-001 appears first and expires at minute 90. D-002 appears second and expires at minute 30. Each has 6 units and is ready at minute 10. One recipient has 10 units of space, and the driver can carry either donation. Only one fits.

Result	FIFO	Greedy
Assigned donation	D-001	D-002
Unassigned donation	D-002	D-001
Donations assigned	1	1
Food quantity rescued	6 units	6 units
Food quantity at risk	6 units	6 units
Urgent rescue rate	0%	100%

Greedy raises urgent rescue rate by 100 percentage points in this test. It protects the food that expires first, while the earlier donor loses their chance. It does not rescue more total food in this case. The sample and test show why we compare both total food saved and which donations get assigned.

Architecture

File	What it does
`main.py`	Loads and checks data, runs both strategies, and prints results.
`data_loader.py`	Reads JSON and checks fields, types, IDs, numbers, and availability times.
`matching.py`	Checks whether food, a recipient, and a driver can match. Updates space and pickup limits after a match.
`algorithms.py`	Runs FIFO and greedy, explains greedy order, and compares three result totals.
`metrics.py`	Counts results and calculates rates using the original input capacities.
`output.py`	Prints assignments, rejection reasons, priority reasons, and the comparison.
`data/`	Holds sample donations, recipients, and volunteers.
`tests/ test_rescueroute.py`	Checks normal cases, edge cases, metrics, and strategy differences.

The steps are: load data, check data, run each strategy, calculate results, and print them. Each strategy uses its own copies of the records. Running FIFO does not use up the resources available to greedy.

Records are dictionaries inside lists. List order gives FIFO its arrival order and decides which recipient and driver are checked first. Sets help find duplicate IDs and count assigned donations once. Greedy uses a sorted list. Metrics uses a dictionary keyed by donation ID to look up food quantities. Accepted food types are stored in a list in the current code.

Data Model

Times are whole minutes from the start of the model. Quantities and capacities use the same made-up food unit. Text values must not be blank.

Donations: `data/donations.json`

Field	Type	Meaning
`donation_id`	Text	Donation ID, unique within the donation list.
`donor_name`	Text	Name of the donor.
`food_type`	Text	Food category, such as `prepared_meals` or `produce`.
`quantity`	Whole number above zero	Amount of food.
`expiry_time`	Whole number, zero or higher	Minute when the food expires.
`pickup_area`	Text	Area where food is picked up.
`ready_time`	Whole number, zero or higher	First minute food can be picked up.

Recipients: `data/recipients.json`

Field	Type	Meaning
`recipient_id`	Text	Recipient ID, unique within the recipient list.
`organization_name`	Text	Name of the group receiving food.

`accepted_food_types`	List of text values	Food categories the group accepts. Each value must not be blank.
`capacity`	Whole number, zero or higher	Total food units the group can receive in this run.
`area`	Text	Area served by the group.
`closing_time`	Whole number, zero or higher	Minute when the group closes.

Volunteers: `data/volunteers.json`

Field	Type	Meaning
`volunteer_id`	Text	Driver ID, unique within the volunteer list.
`vehicle_capacity`	Whole number, zero or higher	Most units the vehicle can carry on one pickup.
`area`	Text	Area served by the driver.
`available_from`	Whole number, zero or higher	First minute the driver is available.
`available_until`	Whole number, zero or higher	End of driver availability. Must be at least `available_from`.
`max_pickups`	Whole number, zero or higher	Most pickups the driver can make in this run.

Food types and areas must match exactly, including spelling and case. There is no fixed list of allowed category names. IDs are checked as

written; spaces are not removed. The current sample contains a trailing space in `D-008 `.

Assignment records: created by the program

Field	Meaning
`donation_id`	Donation being considered.
`recipient_id`	Selected recipient, or `None` when unassigned.
`volunteer_id`	Selected driver, or `None` when unassigned.
`scheduled_time`	Pickup minute, or `None` when unassigned.
`decision_reason`	Reason for the assignment or rejection.
`status`	`assigned` or `unassigned`.
`scheduling_reason`	Extra field in greedy results explaining the priority order.

Simulation assumptions and matching rules

- A donation is assigned as a whole. It cannot be split among recipients or drivers.
- The recipient must accept the food type. The donation, recipient, and driver must all be in the same area.
- The recipient must have enough remaining space for the full donation.
- The vehicle must hold the full donation, and the driver must have a pickup left.
- Pickup happens at the later of the donation's ready time and the driver's available time.
- Pickup must be strictly before food expiry, recipient closing, and the end of driver availability. Pickup exactly at any of those times is rejected.
- There is no travel, loading, unloading, or service time. Pickup and

delivery are treated as happening at the same time. A driver can make several pickups at the same minute if their pickup limit allows it.
- After a match, recipient capacity drops by the food quantity. The driver's pickup limit drops by one, and their available time becomes the pickup time.
- Vehicle capacity stays the same because it applies to each pickup. Recipient space is not refilled during a run.
- Both strategies choose the first matching recipient and driver in input order. They do not search for the best route or best set of matches.
- Expired donations and donations ready at or after expiry are allowed as input. Matching rejects them when no valid pickup time exists.
- Each donation gets one result per run. Unassigned donations are not saved for another run.

FIFO and Greedy rules

FIFO means first in, first out. It considers donations in input order. For each one, it chooses the first recipient and driver that pass the matching checks.

Greedy considers donations in this order:

1. Earlier expiry first.
2. If expiry times match, larger quantity first.
3. If quantities also match, earlier ready time first.
4. If all three values match, keep input order.

Greedy uses the same matching checks and resource updates as FIFO. It gives food with less time left the first chance at limited space and pickups. The quantity rule tries to save more food when expiry times match. Neither strategy goes back to change an earlier choice.

Limits and fairness

Greedy chooses for the next donation. It does not promise the best result for the whole set. For example, with 10 units of recipient space, an urgent 6-unit donation can leave too little room for two later 5-unit donations. Saving those two instead would rescue 10 units, but leave the urgent food unassigned.

The first matching recipient or driver may also be needed by a later donation that has fewer choices. The program does not move assignments to make room.

FIFO favors earlier arrivals. Greedy favors earlier expiry and, when tied, larger donations. The fairness test gives a recipient 6 units of space and two donations with the same expiry: an earlier 4-unit donation and a later 6-unit donation. FIFO assigns the 4-unit donation; greedy assigns the 6-unit donation. The earlier donor loses their chance under greedy, even though their donation could have been assigned on its own.

If a future version keeps waiting donations across runs, repeated urgent arrivals could keep pushing other donations back. Never getting a turn is called starvation. The current program has no waiting queue, so this test shows a fairness trade-off in one run, not starvation over several runs. Giving waiting donations more priority could help, but could delay urgent food.

Metric definitions

Capacity totals use the original inputs, before scheduling. Rates are percentages. A rate with a zero total to divide by is 0%.

Metric	How it is calculated
Donations received	Number of donation records.
Donations assigned	Number of different donation IDs successfully assigned.
Donations unassigned	Received count minus assigned count.

Food quantity rescued	Total quantity of assigned donations.
Food quantity at risk	Total quantity of unassigned donations, including expired ones.
Urgent rescue rate	Assigned donations in the earliest-expiring group divided by all donations in that group, times 100. This group includes every donation tied for the lowest expiry time.
Recipient capacity utilization	Rescued quantity divided by the sum of all original recipient capacities, times 100. This is the share of recipient space used.
Volunteer capacity utilization	Rescued quantity divided by the sum of each driver's vehicle capacity times their maximum pickups, times 100. This is the share of total allowed carrying space used.
FIFO vs. greedy difference	Greedy minus FIFO for assigned count, rescued quantity, and urgent rescue rate. Rate differences are in percentage points.

Capacity rates include all listed recipients and drivers, even ones that cannot serve a particular donation. They do not measure driver working time.

Algorithms and complexity

D means donations, **R** means recipients, and **V** means volunteers. Time means how much work a function does as input grows. Space means how much extra memory it uses. O(1) means the amount stays the same as input grows.

For these bounds, assume records, IDs, and individual check messages have a limited size and each recipient has a small, fixed number of accepted food types. Otherwise, checking food lists and building long messages adds work.

Function	Worst-case time, with at least one recipient and driver
`schedule_fifo`	O(D + R + V + D * R * V)
`greedy_priority`	O(1)
`schedule_greedy`	O(D log D + D + R + V + D * R * V)
`calculate_metrics`	O(D + R + V), since the scheduler returns one result per donation.
`compare_strategies`	O(1)

FIFO can check every recipient with every driver for each donation. This gives up to D * R * V checks. Finding a match early means fewer checks. Copying the records takes O(D + R + V) work and memory. Updating an assignment can search through R recipients and V drivers, taking O(R + V) time.

Greedy first sorts donations, adding O(D log D) time and O(D) memory. It then does the same matching work as FIFO and adds a priority reason to each result. Comparing three totals takes O(1); calculating them is separate.

Both schedulers keep input copies and one result per donation. They also collect up to R * V rejection reasons for a donation. Reasons can name different recipients and drivers, so saved explanation text can grow with failed checks. A safe upper bound for extra memory, including results and messages, is O(D + R + V + D * R * V).

With no drivers, FIFO still visits each recipient for each donation. A time bound covering empty lists too is O(D + R + V + D * (R * V + R + V)); greedy adds its sorting cost.

Team contributions

| Member | Role based on recorded work |

- Brian Nguyen | Algorithms, plus shared matching, testing, and

writing work | Built FIFO, greedy, the priority rule, and strategy comparison; wrote complexity and fairness notes; completed matching and validation; connected the program; added output and tests; fixed volunteer capacity calculations.
- Jimmy | Data loading and sample data | Added JSON loading, early validation and ID checks, and more sample donors and volunteers.
- Dawood | Metrics, clear output, test updates, and demo | Created the repository; added counts and rates; expanded sample data; improved assignment and rejection messages; added greedy priority reasons and a test for them; created demo and any relevant additional deliverables

References and sources

- **Course brief:** *CPSC 335 - RescueRoute Group Project Brief*. Sections 3-8 cover data, algorithms, explanations, metrics, and tests. Sections 11 and 13 cover submission rules and source disclosure.
- **Project code and history:** GitHub repository. Comparison numbers come from running the code; the contribution table links to the commits used to write it.
- **Python documentation:** JSON files, unittest, copying records, and sorting. These explain the built-in tools used by the code and were checked while writing this README.
- **Sample data:** The JSON records are made-up project examples. No outside dataset is listed in the reviewed commit history. No outside packages are needed by the current code.

AI assistance disclosure

AI was used to help write the six previously unfinished automated tests: insufficient recipient capacity, unavailable or undersized volunteers, competing donations, empty input, FIFO-versus-greedy differences, and fairness. This assistance included shared test helpers, edge cases, and assertions for expected assignments, rejection explanations, metrics,

and unchanged input data. Codex ran the full test suite after these changes; all 12 tests passed with no skipped tests.

AI also helped write this README, including setup steps, field descriptions, model rules, metric definitions, the comparison report, and the contribution table based on the repository's commits. The program and tests were run again to check the documented results. Team members remain responsible for reviewing, understanding, and verifying the submitted code, tests, and writing, and for following the course's AI-use policy.

Please Note: AI platforms such as (Gemini, ChatGPT), were used to explain certain concepts within the project, this may include situations where students were shown sample code on how to implement certain features.